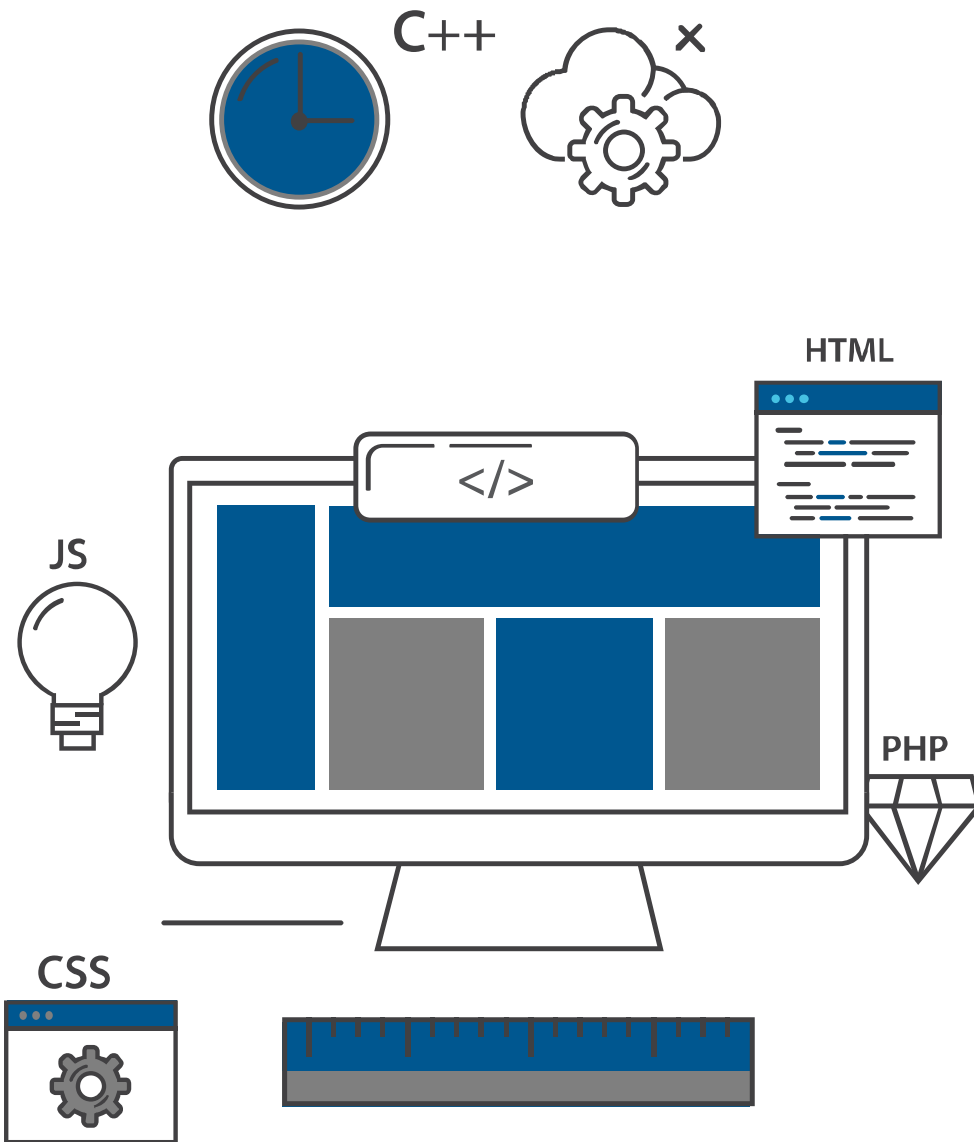
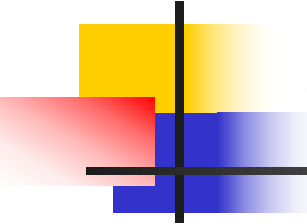


第五章 分治策略

李翔

<https://implus.github.io/>





分治策略

- 经常使用的办法:
- 把一个规模为 n 的问题分解成两个规模 $\frac{1}{2}n$ 的问题;
- 分别递归解决两个部分的问题;
- 在**线性时间**内把这两个部分问题的解合成一个完整的解。

归并排序

■ 归并排序

- Divide array into two halves.
- Recursively sort each half.
- Merge two halves to make sorted whole.



Jon von Neumann (1945)

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	L	G	O	R
---	---	---	---	---

I	T	H	M	S
---	---	---	---	---

divide $O(1)$

A	G	L	O	R
---	---	---	---	---

H	I	M	S	T
---	---	---	---	---

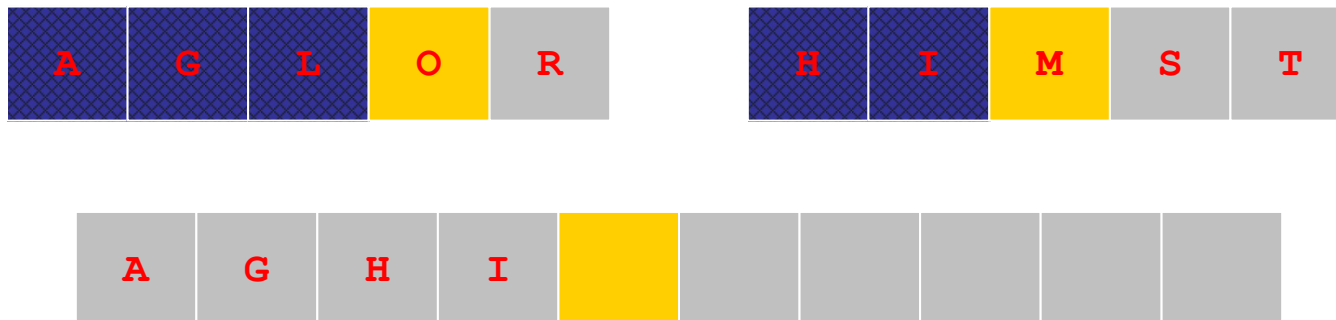
sort $2T(n/2)$

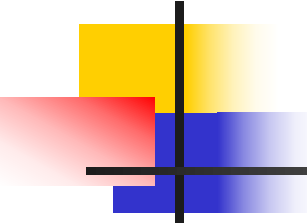
A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

merge $O(n)$

归并排序

- 合并：把两个排好序的部分合成一个完整的排序
- 效率分析：采用一个临时数组；需要 $O(n)$ 次比较





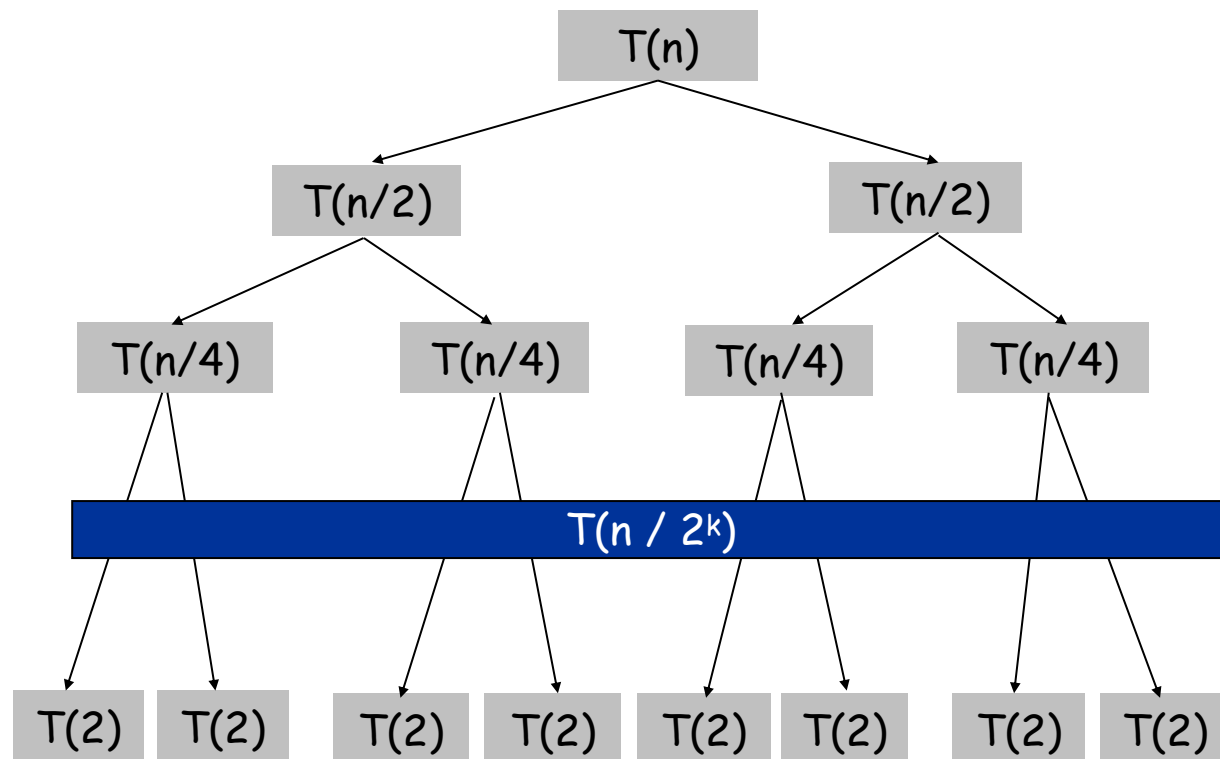
递推关系

- 命题5.1 对某个常数 c ,

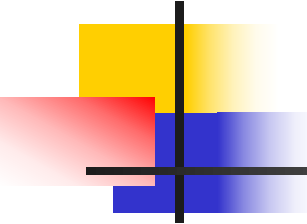
$$T(n) \leq \begin{cases} 2T(n/2) + cn, & n > 2 \\ c, & n = 2 \end{cases}$$

- 或者
$$T(n) \leq \begin{cases} 0 & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \begin{matrix} n \\ \text{merging} \end{matrix} & \text{otherwise} \end{cases}$$

递推关系



$$\begin{array}{c} \uparrow \\ n \\ 2(n/2) \\ 4(n/4) \\ \log_2 n \\ \dots \\ 2^k (n / 2^k) \\ \dots \\ n/2 (2) \\ \hline n \log_2 n \end{array}$$



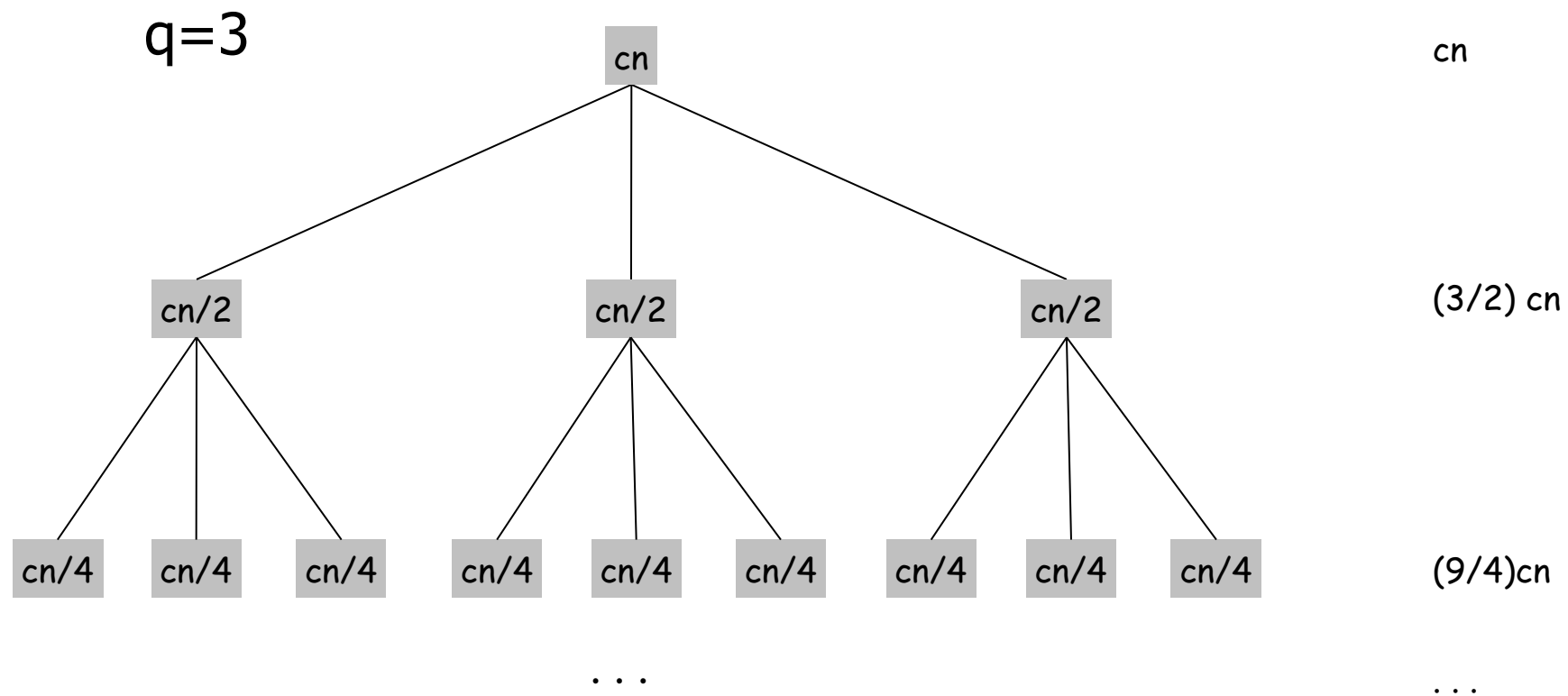
其他递推关系

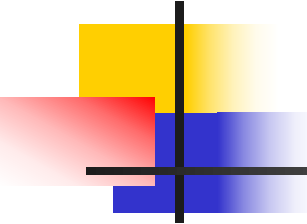
- 命题5.3 对某个常数c,

$$T(n) \leq \begin{cases} qT(n/2) + cn, & n > 2 \\ c, & n = 2 \end{cases}$$

$q > 2$, $q = 1$, $q = 2$ 递推关系的性质不同

其他递推关系



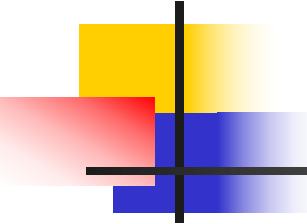


其他递推关系

类似的根据上图可以得到:

$$\begin{aligned} T(n) &\leq \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j cn = cn \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j = cn \left(\frac{r^{\log_2 n} - 1}{r - 1} \right) \\ &\leq cn \left(\frac{r^{\log_2 n}}{r - 1} \right) = \frac{c}{r - 1} n \cdot r^{\log_2 n} = \frac{c}{r - 1} n \cdot n^{\log_2 r} = \frac{c}{r - 1} n^{\log_2 q} = O(n^{\log_2 q}) \end{aligned}$$

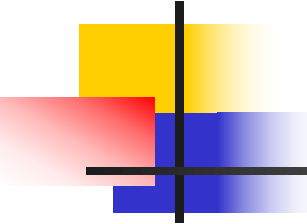
定理5.4 任何满足5.3并具有 $q > 2$ 的函数 $T(\cdot)$ 是 $O(n^{\log_2 q})$ 有界的。



其他递推关系

- 定理5.5 任何满足5.3式并具有 $q=1$ 的函数 $T(\cdot)$ ($T(n) \leq T(n/2) + O(n)$) 是 $O(n)$ 有界的。
- 证明:

$$T(n) \leq \sum_{j=0}^{\log_2 n - 1} \frac{cn}{2^j} = cn \sum_{j=0}^{\log_2 n - 1} \frac{1}{2^j} \leq 2cn = O(n)$$



其他递推关系

- 命题5.6 对某个常数c,

$$T(n) \leq \begin{cases} 2T(n/2) + cn^2, n > 2 \\ c, n = 2 \end{cases}$$

T(n)的复杂度应该是?

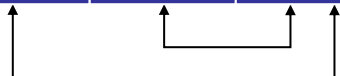
$$T(n) = O(n^2)$$

计数逆序

- 相似度量?
- 考虑逆序的个数
- 两个指标 $i < j$ 构成一个逆序, 如果 $a_i > a_j$.

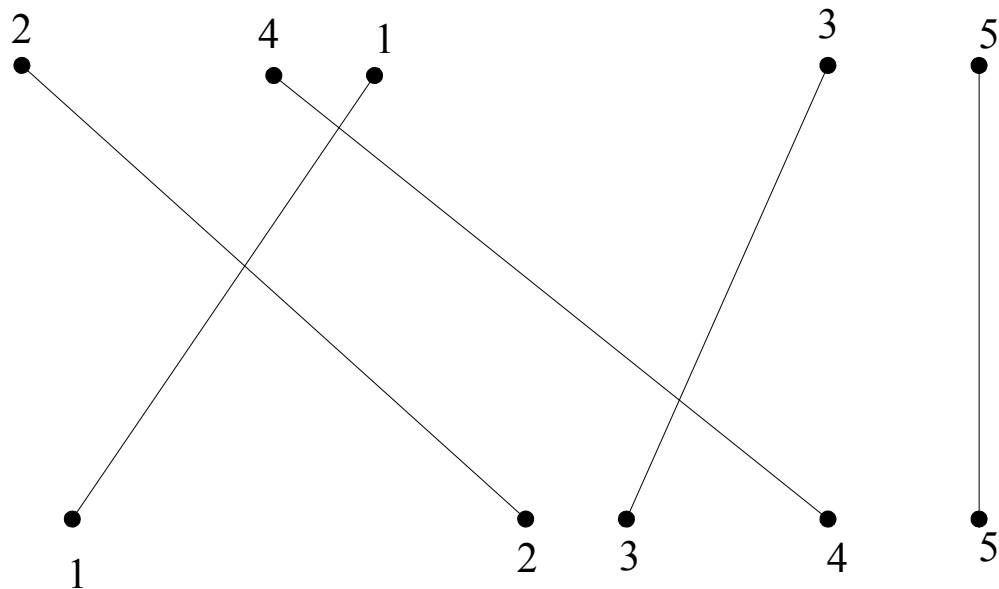
Songs

	A	B	C	D	E
Me	1	2	3	4	5
You	1	3	4	2	5

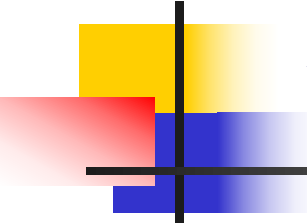


Inversions
3-2, 4-2

计数逆序



逆序数: 3---(2,1),(4,1),(4,3)



计数逆序

- 穷举 (Brute force) : $\Theta(n^2)$.

- 排序: $\Theta(n \lg n)$

- ? 分治策略

---可同时进行逆序计数的工作

例子

■ 分治策略

- 划分：把序列一分为二；
- 分别递归求出左右各自部分的逆序数；
- **组合**：计数左右部份之间的逆序数, 返回总的逆序数.

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

Divide: $O(1)$.

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

Conquer: $2T(n / 2)$

5 blue-blue inversions

8 green-green inversions

9 blue-green inversions

5-3, 4-3, 8-6, 8-3, 8-7, 10-6, 10-9, 10-3, 10-7

Total = $5 + 8 + 9 = 22$.

Efficient Combine: ???

例子

- 组合：计数蓝-绿逆序，**假设左右部分已经排序**，合并这两个部分为完整的排序。

3	7	10	14	18	19
---	---	----	----	----	----

2	11	16	17	23	25
6	3	2	2	0	0

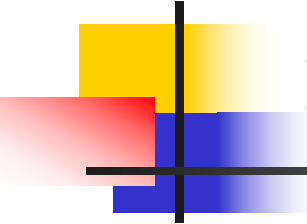
13 blue-green inversions: $6 + 3 + 2 + 2 + 0 + 0$

Count: $O(n)$

2	3	7	10	11	14	16	17	18	19	23	25
---	---	---	----	----	----	----	----	----	----	----	----

Merge: $O(n)$

$$T(n) \leq T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + O(n) \Rightarrow T(n) = O(n \log n)$$



算法

Merge-and-Count(A,B)

维护一个Current指针指向每个表，初始化指向首元素

维护一个变量Count用于逆序的计数，初始为0

While两个表都不空

 令 a_i 与 b_j 是由Current指针指向的元素

 把这两个表中较小的元素加到输出表中

 If b_j 是较小的元素 then

 把Count加上在A中剩余的元素数

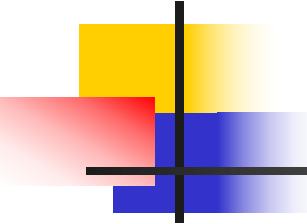
 Endif

 把较小元素选出的表中的Current指针前移

Endwhile

一旦一个表为空，把另一个表剩余的所有元素加到输出中

返回Count和合并后的表



算法

Sort-and-Count(L)

If 这个表中有一个元素, then
没有逆序

Else

把这个表划分成两半,

A 包含前 $\lceil n/2 \rceil$ 个元素

B 包含剩下的 $\lfloor n/2 \rfloor$ 个元素

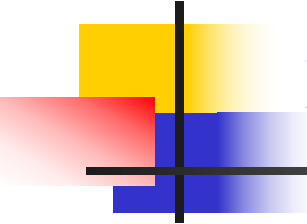
$(r_A, A) = \text{Sort-and-Count}(A)$

$(r_B, B) = \text{Sort-and-Count}(B)$

$(r, L) = \text{Merge-and-Count}(A, B)$

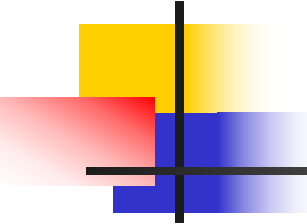
Endif

返回 $r = r_A + r_B + r$ 以及排好序的表 L



算法分析

- 定理5.7 Sort-and-Count算法正确对输入表排序并且计数逆序个数；它对具有 n 个元素的表运行在 $O(n \log n)$ 时间。

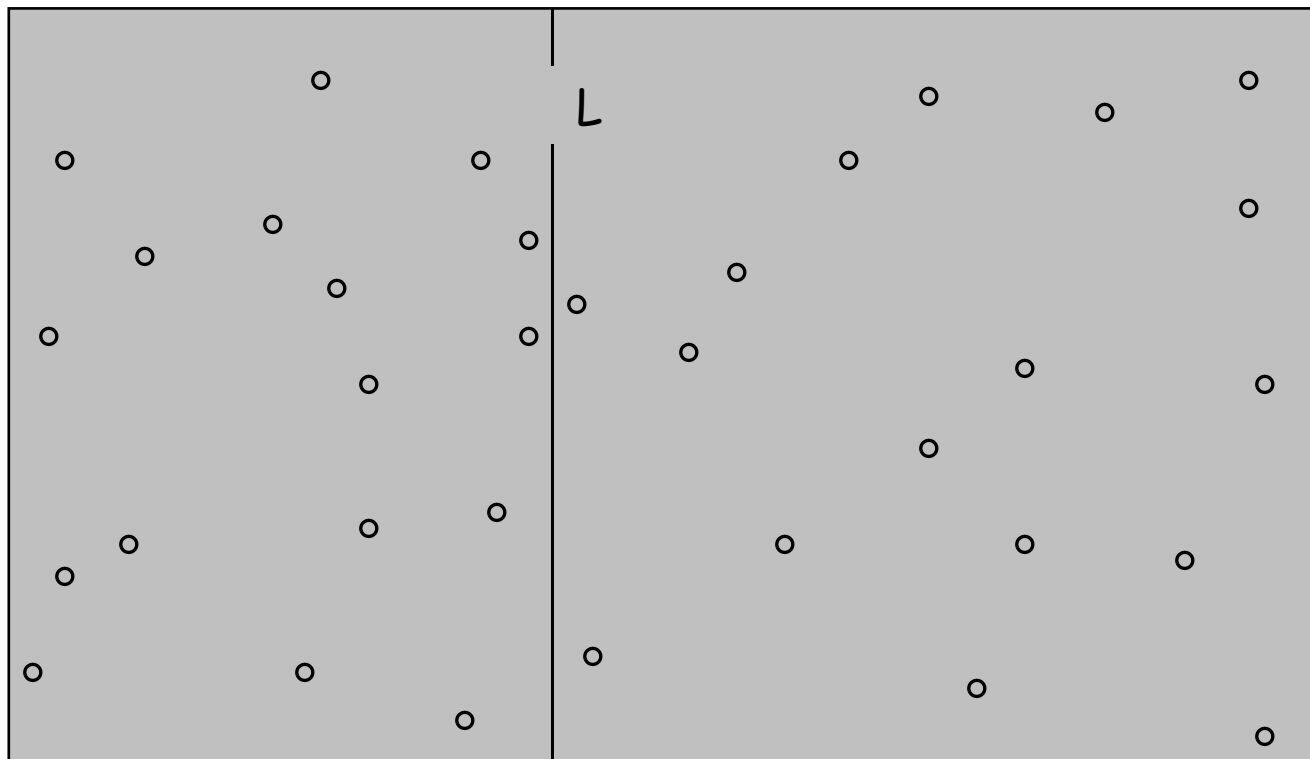


5.4 最邻近点对

- 问题：给定平面上的 n 个点，找最邻近的一对点
- 计算几何领域的基础
- 20世纪70年代， M. I. Shamos, D. Hoey,
关心是否能找到一个渐进的比平方阶更快的算法

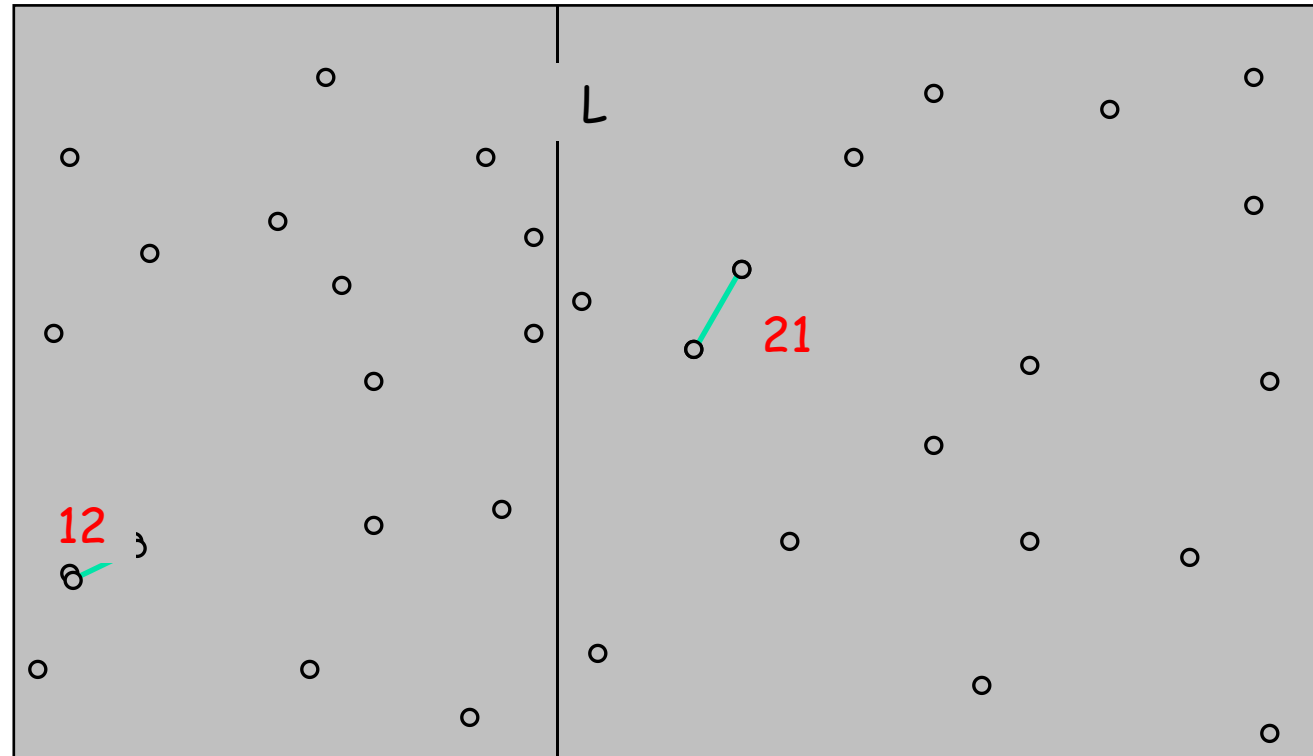
分治算法

- 划分的时候：用一条垂直线把点集分成相等的两部分



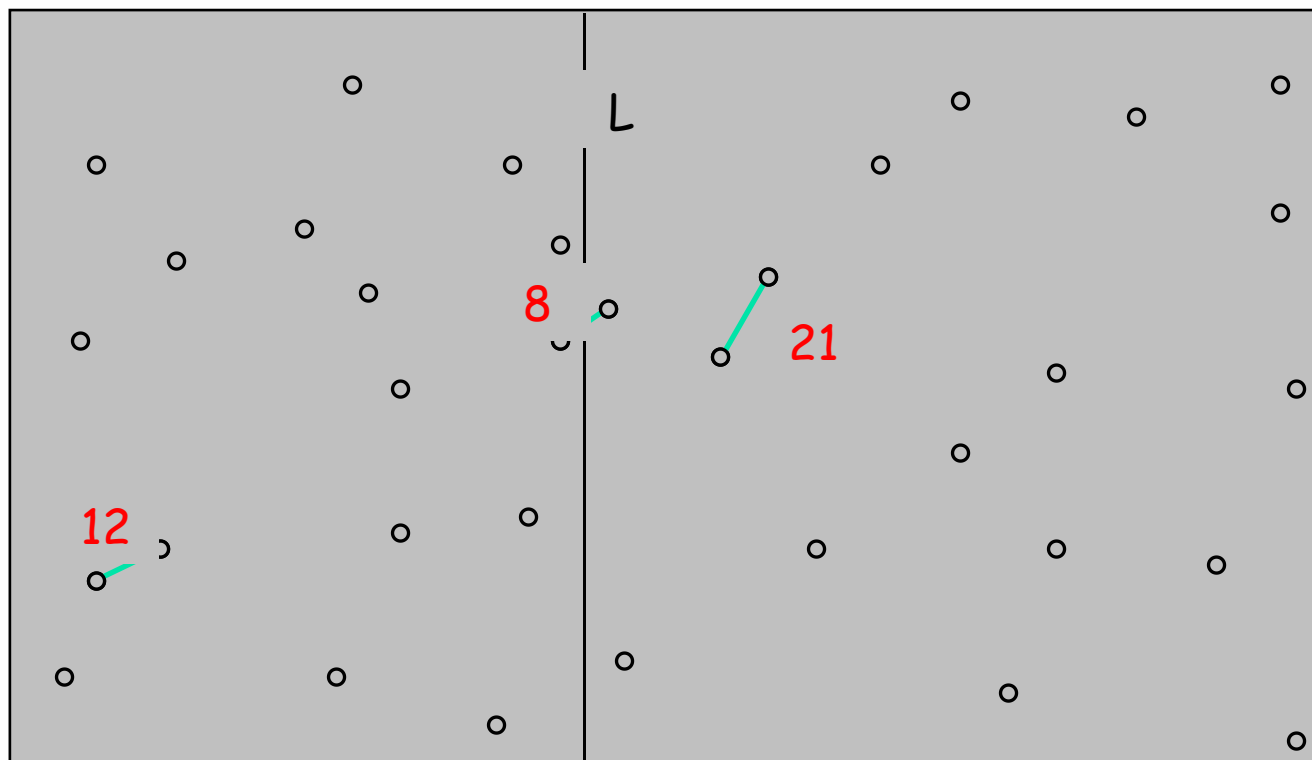
分治算法

- 在左右半部分寻找各自最邻近的点对



分治算法

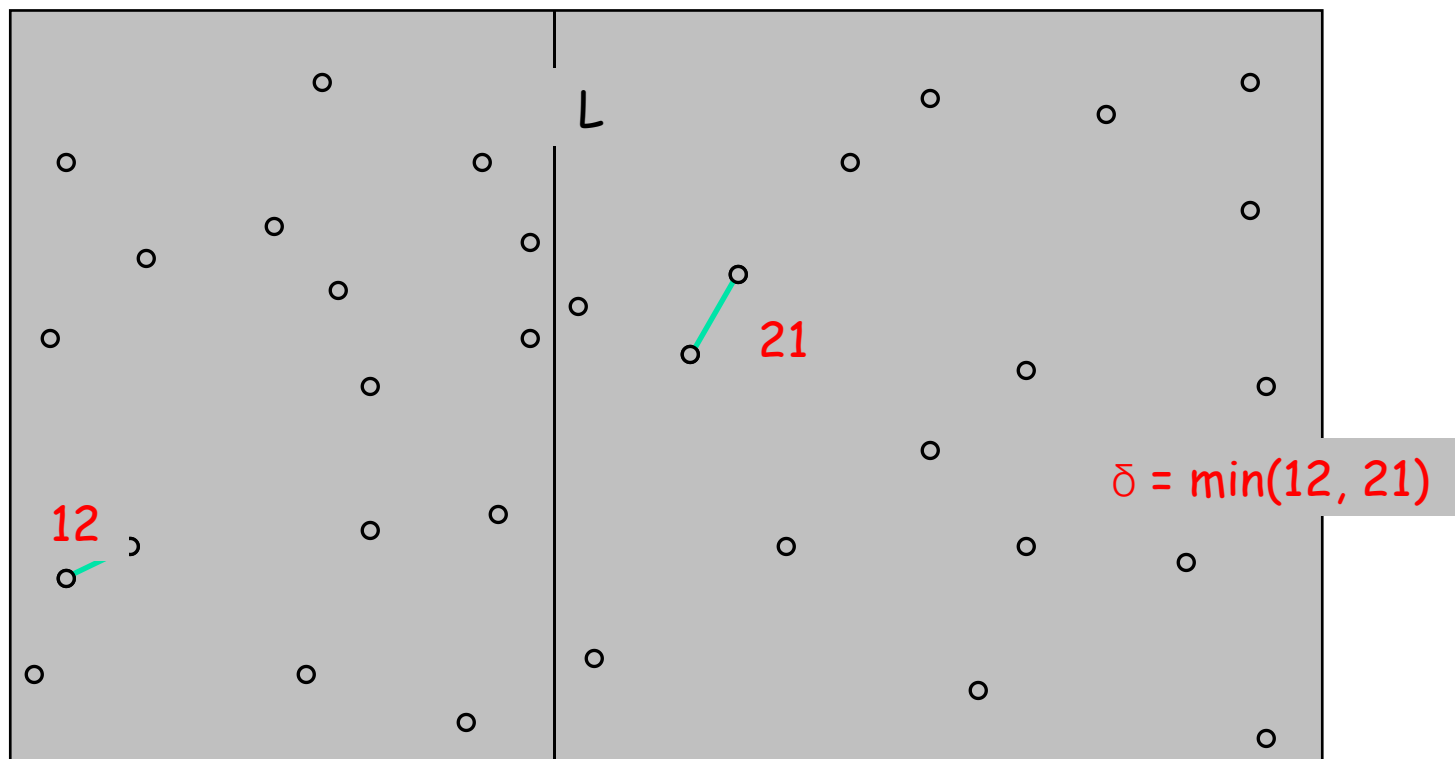
- 组合：在“交界”的边中寻找最近的点对
- 最后返回上面3个解中的最优解



分治算法

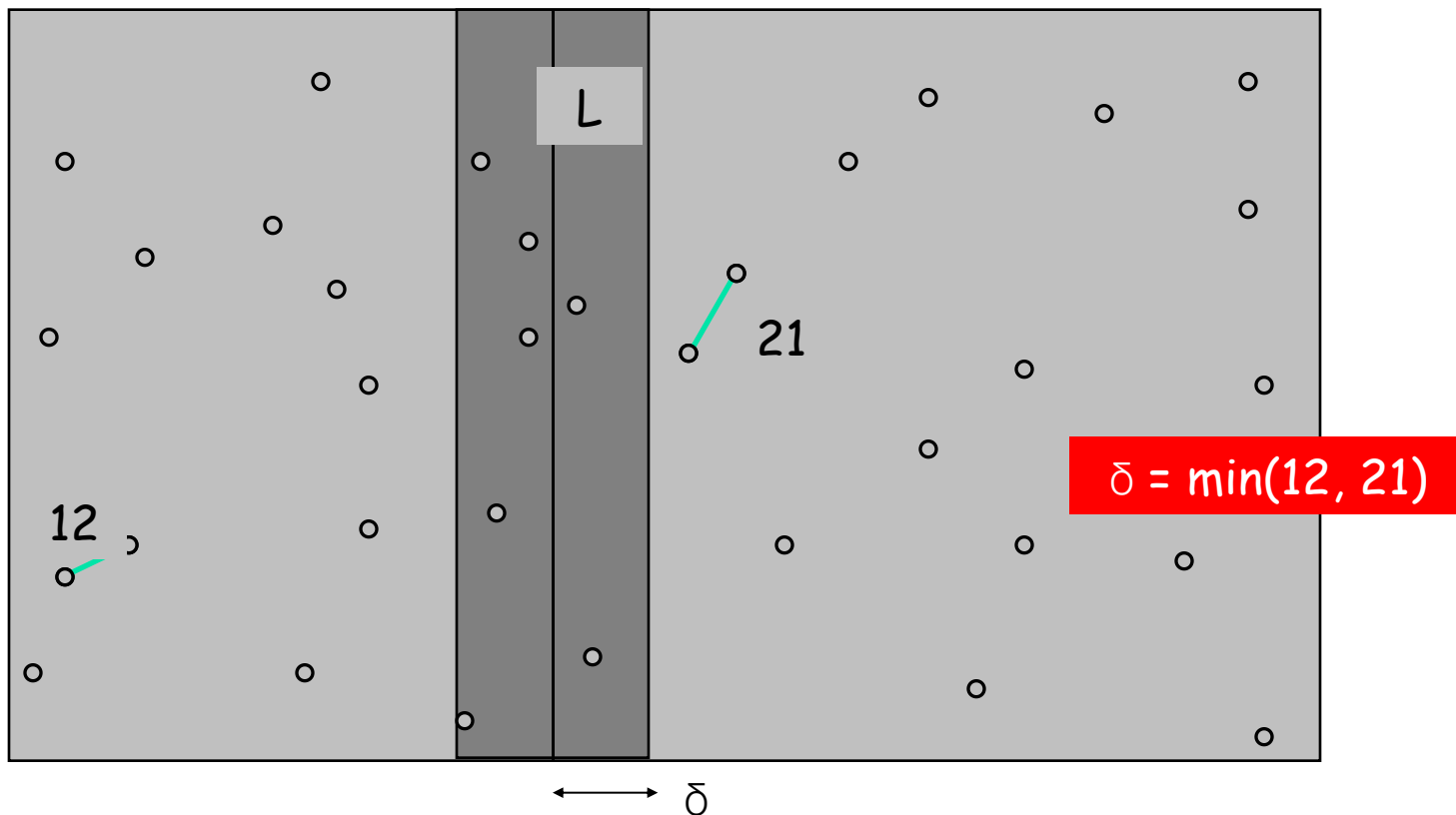
- 问题的关键在于：寻找“交界”的边中比还要近的点。

$$\delta = \min(\delta_L, \delta_R)$$



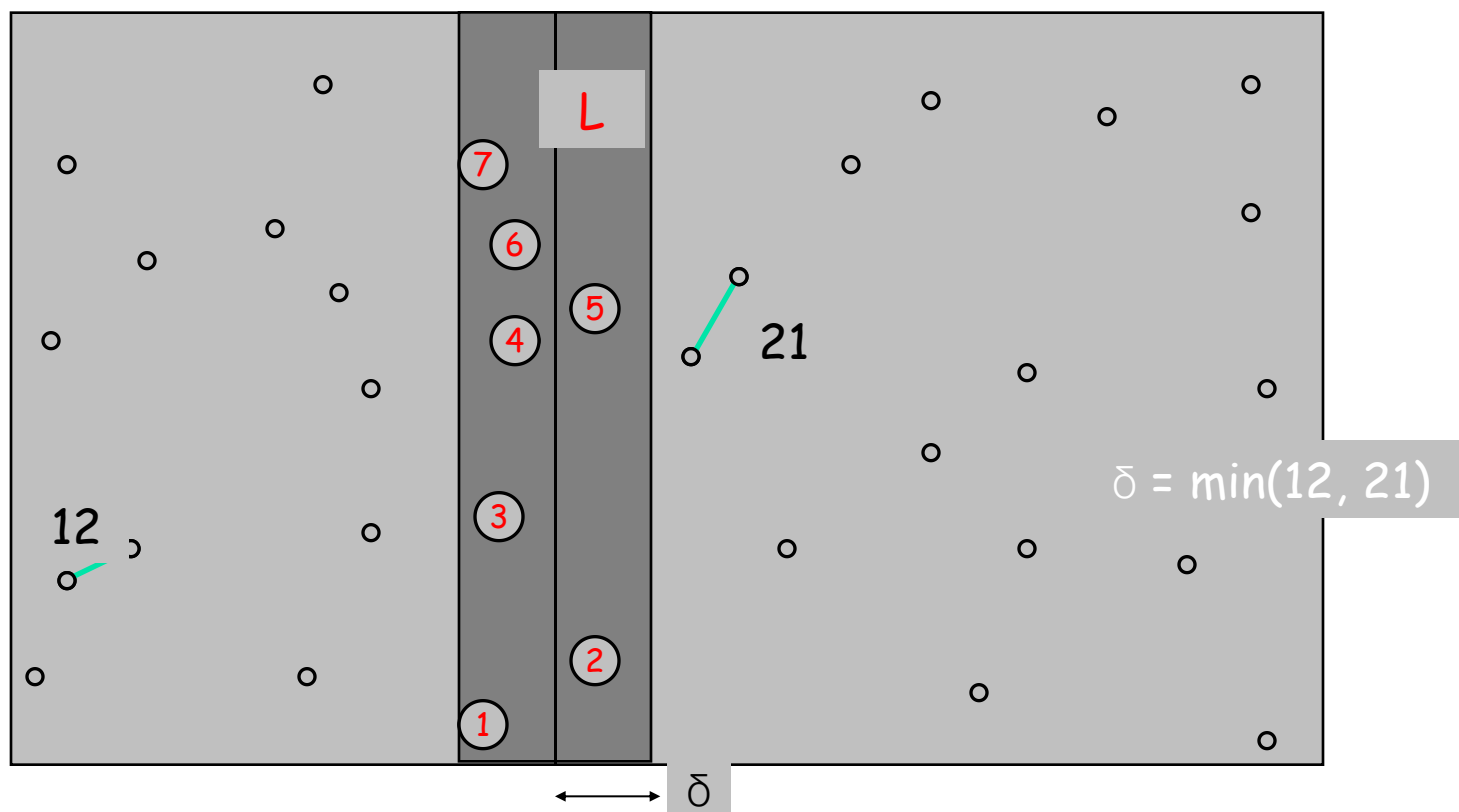
分治算法

- 于是根据平面几何的约束性质，只需要考虑分界线附近的点集情况。



分治算法

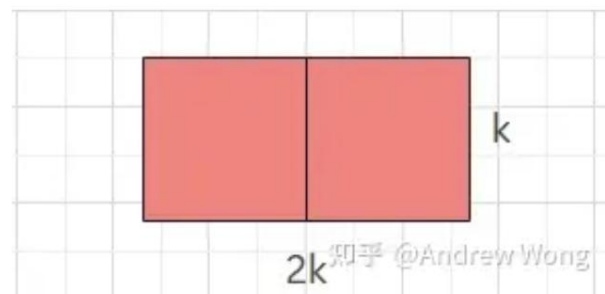
- 把“窄带”之中的点集按照y坐标的大小排序编号



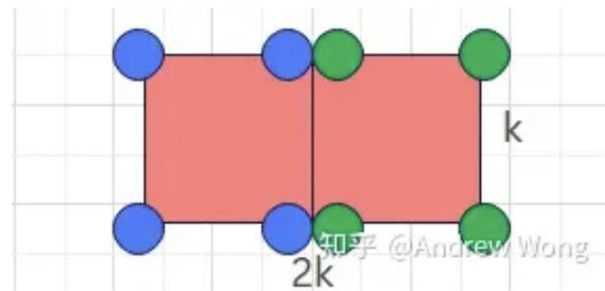
不需要在窄带之间进行两两之间点的比较

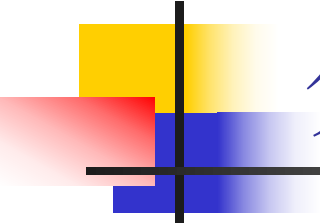
注意到这里目的是为了说明不需要在窄带之间进行两两之间点的比较；只需要在一个常数阶的范围内比较就可以。

现在我们考虑一个问题，对于下图中的红色区域，由上述划分可知，每个正方形区域中任意两点之间的距离不小于 k ，那么两个正方形区域中最多可以容纳多少点？



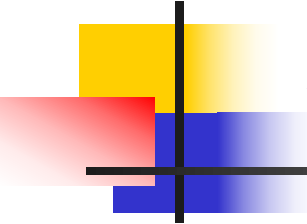
如下图所示，最多只能容纳8个点，8个点分布在两个正方形的四角。





分治算法

```
Closest-Pair( $p_1, \dots, p_n$ ) {  
    Compute separation line  $L$  such that half the points  
    are on one side and half on the other side.  $O(n \log n)$   
  
     $\delta_1 = \text{Closest-Pair}(\text{left half})$   
     $\delta_2 = \text{Closest-Pair}(\text{right half})$   $2T(n / 2)$   
     $\delta = \min(\delta_1, \delta_2)$   
  
    Delete all points further than  $\delta$  from separation line  $L$   $O(n)$   
  
    Sort remaining points by  $y$ -coordinate.  $O(n \log n)$   
  
    Scan points in  $y$ -order and compare distance between  
    each point and next 11 neighbors. If any of these  
    distances is less than  $\delta$ , update  $\delta$ .  $O(n)$   
  
    return  $\delta$ .  
}
```



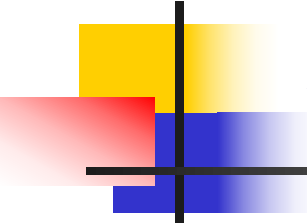
分治算法

- 运行时间:

$$T(n) \leq 2T(n/2) + O(n \log n)$$

复杂度?

$$T(n) = O(n \log^2 n)$$



分治算法

- 是否存在 $O(n \log n)$ 的算法?
- 上述算法在窄带之中的排序消耗了 $O(n \log n)$; 所以希望在窄带中能够不必从头开始排序
- ~~■ 通过预处理, 可以在 $\Theta(n)$ 的时间内实现窄带中的排序~~
- 通过顺便按照y归并排序来实现

<https://oi-wiki.org/geometry/nearest-points/>

我们使用一个结构体来存储点，并定义用于排序的函数对象：

结构体定义

```
1 struct pt {
2     int x, y, id;
3 };
4
5 struct cmp_x {
6     bool operator()(const pt& a, const pt& b) const {
7         return a.x < b.x || (a.x == b.x && a.y < b.y);
8     }
9 };
10
11 struct cmp_y {
12     bool operator()(const pt& a, const pt& b) const { return a.y < b.y; }
13 };
14
15 int n;
16 vector<pt> a;
```

为了方便实现递归，我们引入 `upd_ans()` 辅助函数来计算两点间距离并尝试更新答案：

答案更新函数

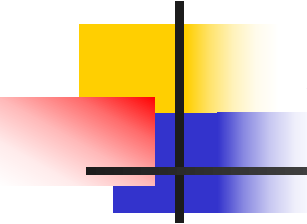
```
1 double mindist;
2 int ansa, ansb;
3
4 void upd_ans(const pt& a, const pt& b) {
5     double dist =
6         sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y) + .0);
7     if (dist < mindist) mindist = dist, ansa = a.id, ansb = b.id;
8 }
```

分治算法

我们使用 `std::inplace_merge()` 来执行归并排序，并创建辅助缓冲区 `t[]`，`B` 存储在其中。

主体函数

```
1 void rec(int l, int r) {
2     if (r - l <= 3) {
3         for (int i = l; i <= r; ++i)
4             for (int j = i + 1; j <= r; ++j) upd_ans(a[i], a[j]);
5         sort(a + l, a + r + 1, &cmp_y);
6         return;
7     }
8
9     int m = (l + r) >> 1;
10    int midx = a[m].x;
11    rec(l, m), rec(m + 1, r);
12    inplace_merge(a + l, a + m + 1, a + r + 1, &cmp_y);
13
14    static pt t[MAXN];
15    int tsz = 0;
16    for (int i = l; i <= r; ++i)
17        if (abs(a[i].x - midx) < mindist) {
18            for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y < mindist; --j)
19                upd_ans(a[i], t[j]);
20            t[tsz++] = a[i];
21        }
22 }
```

分治算法

- 定理5.12 上面修改后的分治算法的运行时间是 $O(n \log n)$.
- 证明:

$$T(n) \leq 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

整数乘法

- 有没有更好的办法?
- 采用分治策略: 每个整数分成高位, 低位

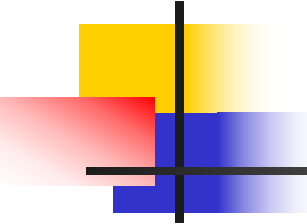
$$x = 2^{n/2} \cdot x_1 + x_0$$

$$y = 2^{n/2} \cdot y_1 + y_0$$

$$xy = (2^{n/2} \cdot x_1 + x_0)(2^{n/2} \cdot y_1 + y_0) = 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0$$

算法有没有改进?

$$T(n) = \underbrace{4T(n/2)}_{\text{recursive calls}} + \underbrace{1 \cdot 2 \cdot 3}_{\text{add, shift}} \Rightarrow T(n) = \Theta(n^2)$$



整数乘法

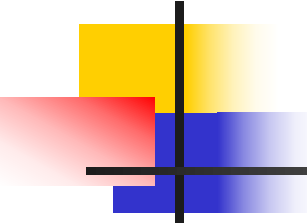
$$x = 2^{n/2} \cdot x_1 + x_0$$

$$y = 2^{n/2} \cdot y_1 + y_0$$

$$xy = 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0$$

$$= 2^n \cdot \underset{A}{x_1 y_1} + 2^{n/2} \cdot \left(\underset{B}{(x_1 + x_0)(y_1 + y_0)} - \underset{A}{x_1 y_1} - \underset{C}{x_0 y_0} \right) + \underset{C}{x_0 y_0}$$

所以可以把4个乘法变成3个!



整数乘法

- Recursive-Multiply(x,y)

令 $x = x_1 2^{n/2} + x_0, y = y_1 2^{n/2} + y_0$

计算 $x_1 + x_0$ 与 $y_1 + y_0$

$p = \text{Recursive-Multiply}(x_1 + x_0, y_1 + y_0)$

$x_1 y_1 = \text{Recursive-Multiply}(x_1, y_1)$

$x_0 y_0 = \text{Recursive-Multiply}(x_0, y_0)$

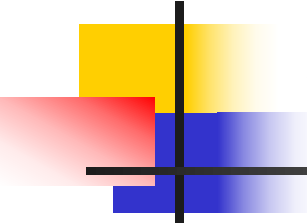
Return $x_1 y_1 2^n + (p - x_1 y_1 - x_0 y_0) 2^{n/2} + x_0 y_0$

分析算法

- 定理5.13 [**Karatsuba-Ofman, 1962**] Recursive-Multiply 算法在两个n位因数上的运行时间是 .
- 证明: $(n^{\log_2 3}) = O(n^{1.59})$

$$T(n) \leq \underbrace{T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + T(1 + \lceil n/2 \rceil)}_{\text{recursive calls}} + \underbrace{1 + 4 + 2 + 4}_{\text{add, subtract, shift}} \cdot \frac{n}{4}$$

$$\Rightarrow T(n) = O(n^{\log_2 3}) = O(n^{1.585})$$



矩阵乘积

- 问题：给定两个 $n \times n$ 矩阵 A, B , 求 $C=AB$.

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$
$$\begin{bmatrix} c_{11} & c_{12} & \text{L} & c_{1n} \\ c_{21} & c_{22} & \text{L} & c_{2n} \\ \text{M} & \text{M} & \text{O} & \text{M} \\ c_{n1} & c_{n2} & \text{L} & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \text{L} & a_{1n} \\ a_{21} & a_{22} & \text{L} & a_{2n} \\ \text{M} & \text{M} & \text{O} & \text{M} \\ a_{n1} & a_{n2} & \text{L} & a_{nn} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & \text{L} & b_{1n} \\ b_{21} & b_{22} & \text{L} & b_{2n} \\ \text{M} & \text{M} & \text{O} & \text{M} \\ b_{n1} & b_{n2} & \text{L} & b_{nn} \end{bmatrix}$$

- 如果采用直接计算，那么复杂度为 $\Theta(n^3)$ 次算术运算(arithmetic operations).
- 存不存在更有效的办法？

矩阵乘积

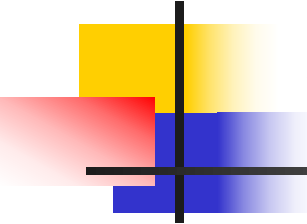
■ 采用分治策略

- 先把A,B分成 $\frac{1}{2}n \times \frac{1}{2}n$ 的小块;
- 分别递归计算8块 $\frac{1}{2}n \times \frac{1}{2}n$ 矩阵乘积;
- 组合: 把结果通过4次矩阵加法组合产生。

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= (A_{11} \times B_{11}) + (A_{12} \times B_{21}) \\ C_{12} &= (A_{11} \times B_{12}) + (A_{12} \times B_{22}) \\ C_{21} &= (A_{21} \times B_{11}) + (A_{22} \times B_{21}) \\ C_{22} &= (A_{21} \times B_{12}) + (A_{22} \times B_{22}) \end{aligned}$$

$$T(n) = \underbrace{8T(n/2)}_{\text{recursive calls}} + \underbrace{4}_{\text{add, form submatrices}} \cdot \underbrace{(n^2)}_{\text{matrix multiplication}} \Rightarrow T(n) = O(n^3)$$



矩阵乘积

- 问题的关键在于能否减少乘法的次数

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

$$P_1 = A_{11} \times (B_{12} - B_{22})$$

$$P_2 = (A_{11} + A_{12}) \times B_{22}$$

$$P_3 = (A_{21} + A_{22}) \times B_{11}$$

$$P_4 = A_{22} \times (B_{21} - B_{11})$$

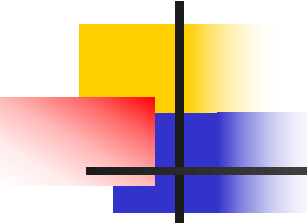
$$P_5 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$$

$$P_6 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$P_7 = (A_{11} - A_{21}) \times (B_{11} + B_{12})$$

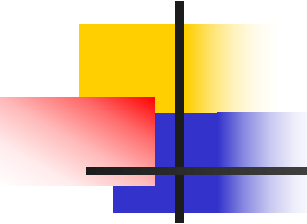
7 次乘法

18 = 10 + 8 次加减法



矩阵乘积

- 矩阵快速乘积运算. (Strassen, 1969)
 - 先把A,B分成 $\frac{1}{2}n \times \frac{1}{2}n$ 的小块;
 - 计算: 通过10次加减法运算, 生成14 个 $\frac{1}{2}n \times \frac{1}{2}n$ 矩阵.
 - 分治: 递归生成7个 $\frac{1}{2}n \times \frac{1}{2}n$ 的矩阵乘积.
 - 组合: 7个矩阵乘积通过8次矩阵的加减法生成需要的四个矩阵乘积



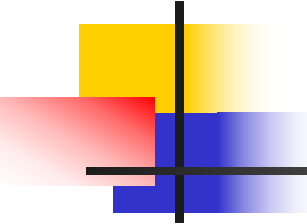
矩阵乘积

■ 分析

- 假设 n 是 2 的整数次幂.
- $T(n) = \#$ arithmetic operations.

$$T(n) = \underbrace{7T(n/2)}_{\text{recursive calls}} + \underbrace{14}_{\text{add, subtract}} \cdot \frac{n^2}{4} \Rightarrow T(n) = O(n^{\log_2 7}) = O(n^{2.81})$$

注意到这是一个理论上的界， n 比较大的时候才能体现出Strassen算法的优势；比如， $n \sim 2,500$ ，Strassen算法在苹果机上的运行速度是常规算法的八十多倍



5.6 卷积与快速傅立叶变换

- 傅立叶变换在数字计算机出现以后，才引起了足够的重视

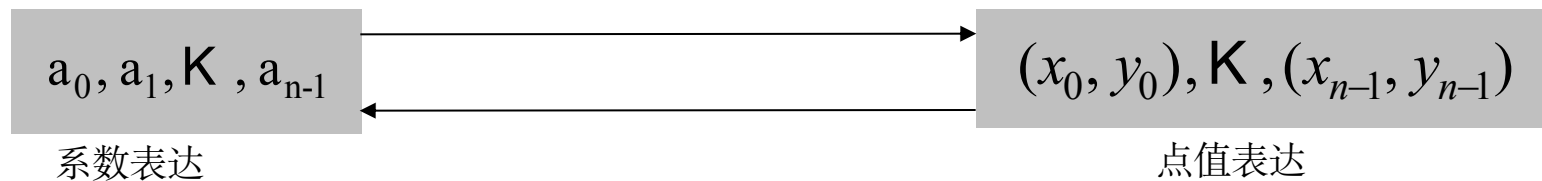
The FFT is one of the truly great computational developments of 20th century. It has changed the face of science and engineering so much that it is not an exaggeration to say that life as we know it would be very different without the FFT. -
Charles van Loan

两种表达之间的关系

- 多项式表达, 点值表达

Representation	乘法	求值
系数表达	$O(n^2)$	$O(n)$
点值表达	$O(n)$	$O(n^2)$

- 希望能够找到一种新的表达方式, 能够在这两种表达之间进行高效的转换



两种表达之间的关系

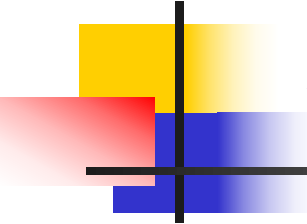
■ 系数表达到点值的表达

给定多项式 $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$, 求它在 n 个不同点 $x_0, \dots, x_{n-1}$ 处的值

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

$O(n^2)$ for matrix-vector multiply

↑
Vandermonde matrix is invertible iff x_i distinct
范德蒙矩阵



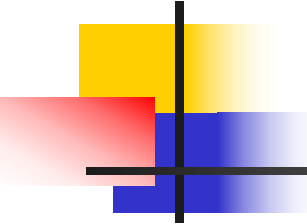
分治策略

- 系数表达到点值表达
- 采用分治策略:

把多项式表达分成奇偶部分

- $A(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7.$
- $A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + a_6x^3.$
- $A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + a_7x^3.$
- **$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$**
- **$A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2).$**

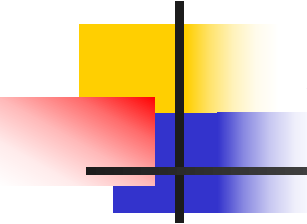
小规模多项式的点值情况已知的情况下,
直接测算出2倍规模问题的点值情况



分治策略

- 尝试：考虑特殊点值 $x=\pm 1$.
 - $A(1) = A_{\text{even}}(1) + 1 A_{\text{odd}}(1)$.
 - $A(-1) = A_{\text{even}}(1) - 1 A_{\text{odd}}(1)$.

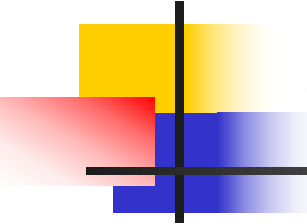
如果计算两个次数小于等于 $\frac{1}{2}n$ 的多项式在一点($x=1$)的值, 那么可以确定一个次数小于等于 n 的多项式在两点的值($x=\pm 1$).



分治策略

- 另一次尝试: 考虑特殊点值 $x=\pm 1, \pm i$.
 - $A(1) = A_{\text{even}}(1) + 1 A_{\text{odd}}(1)$.
 - $A(-1) = A_{\text{even}}(1) - 1 A_{\text{odd}}(1)$.
 - $A(i) = A_{\text{even}}(-1) + i A_{\text{odd}}(-1)$.
 - $A(-i) = A_{\text{even}}(-1) - i A_{\text{odd}}(-1)$.

如果计算2个次数小于等于 $\frac{1}{2}n$ 的多项式在两点($x=1, -1$)的值, 那么可以确定一个次数小于等于 n 的多项式在4点的值($x=\pm 1, \pm i$).



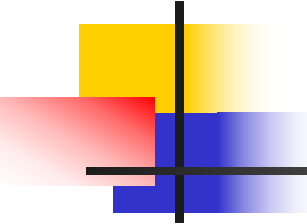
分治策略

系数表达到点值表达：给定多项式 $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$ ，求它在 n 个不同点 $x_0, \dots, x_{n-1}$ 处的值

- 关键之处：选择 $x_k = \omega^k$ ，其中 ω 是 n 次单位根。

分治策略

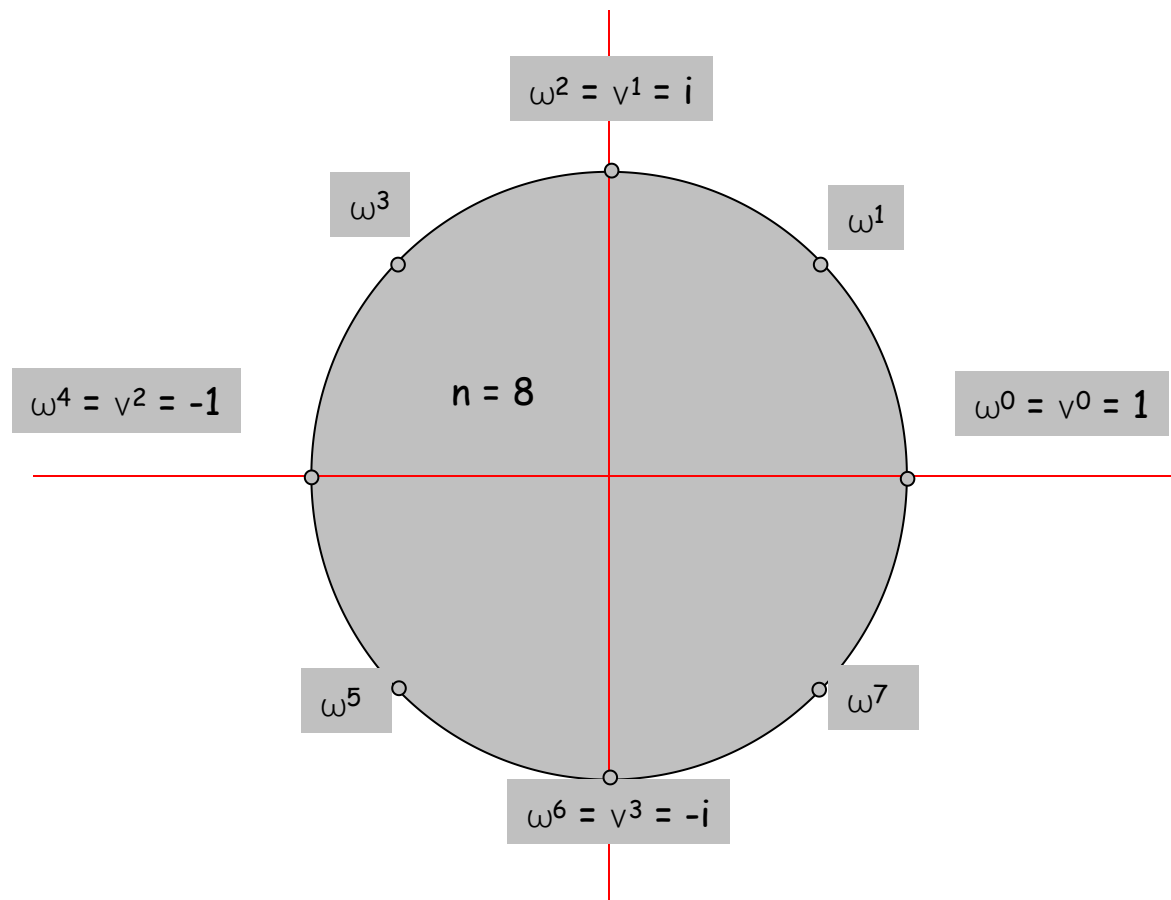
$$\begin{array}{c}
 \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix} \\
 \uparrow \\
 \text{Discrete Fourier transform}
 \end{array}
 =
 \begin{array}{c}
 \begin{bmatrix} 1 & 1 & 1 & 1 & \text{L} & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 & \text{L} & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \text{L} & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \text{L} & \omega^{3(n-1)} \\ \text{M} & \text{M} & \text{M} & \text{M} & \text{O} & \text{M} \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \text{L} & \omega^{(n-1)(n-1)} \end{bmatrix} \\
 \uparrow \\
 \text{Fourier matrix } F_n
 \end{array}
 \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

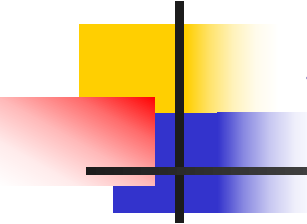


单位根的性质

- 定义：如果复数 x 满足 $x^n = 1$ ，那么 x 被称为 n 次单位根。
- n 次单位根可以表示为 $\omega^0, \omega^1, \dots, \omega^{n-1}$ ，其中 $\omega = e^{2\pi i / n}$ 。
- 如果 n 是偶数，那么 $n/2$ 次单位根可以表示为 $v^0, v^1, \dots, v^{n/2-1}$ ，其中 $v = e^{4\pi i / n}$ 。

单位根的性质





快速傅立叶变换

- 目标: 求 $A(x) = a_0 + \dots + a_{n-1} x^{n-1}$ 在 n 次单位根 $\omega^0, \omega^1, \dots, \omega^{n-1}$ 处的值。
- 分治策略:
- 把多项式分成偶数部分和奇数部分
 - $A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n/2-2} x^{(n-1)/2}.$
 - $A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n/2-1} x^{(n-1)/2}.$
 - $A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$

快速傅立叶变换

- **分别计算**: 计算多项式 $A_{\text{even}}(x)$, $A_{\text{odd}}(x)$ 在 $\frac{1}{2}n^{\text{th}}$ 单位根 $v^0, v^1, \dots, v^{n/2-1}$ 处的值.

- **组合**:

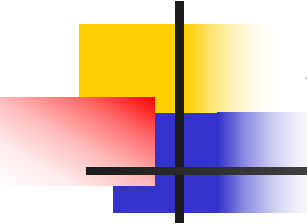
- $A(\omega^k) = A_{\text{even}}(v^k) + \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$

- $A(\omega^{k+n/2}) = A_{\text{even}}(v^k) - \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$

$$\uparrow \\ \omega^{k+n/2} = -\omega^k$$

$$\nwarrow \\ v^k = (\omega^k)^2 = (\omega^{k+n/2})^2$$

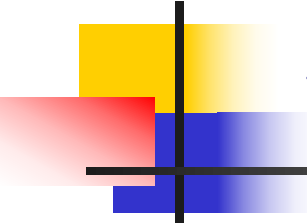
- $A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$



快速傅立叶变换

■ FFT算法

```
fft(n, a0, a1, ..., an-1) {  
    if (n == 1) return a0  
  
    (e0, e1, ..., en/2-1) ← FFT(n/2, a0, a2, a4, ..., an-2)  
    (d0, d1, ..., dn/2-1) ← FFT(n/2, a1, a3, a5, ..., an-1)  
  
    for k = 0 to n/2 - 1 {  
        ωk ← e2πik/n  
        yk ← ek + ωk dk  
        yk+n/2 ← ek - ωk dk  
    }  
  
    return (y0, y1, ..., yn-1)  
}
```

快速傅立叶变换

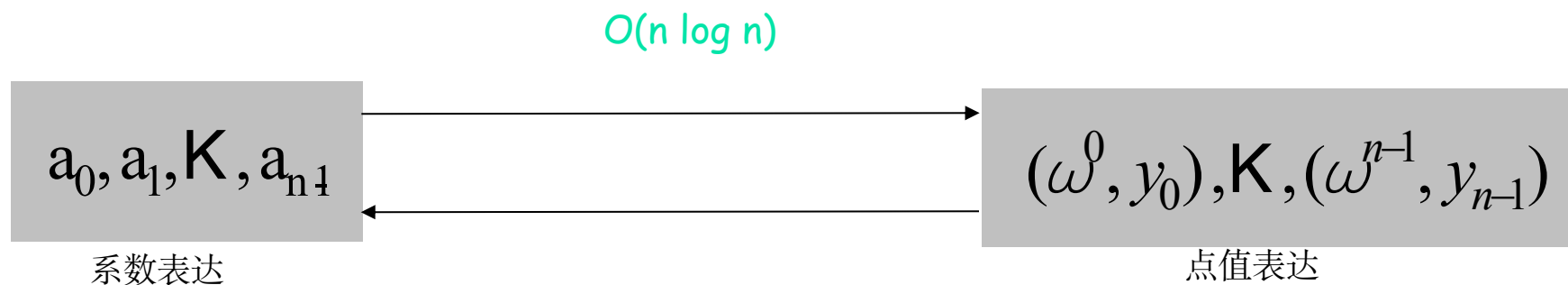
- 定理. FFT 算法能够在 $O(n \log n)$ 步骤内计算 $n-1$ 次多项式在每个 n 次单位根处的值(n 是2的整数次幂)。

- 运行时间

$$T(2n) = 2T(n) + O(n)$$

$$\Rightarrow T(n) = O(n \log n).$$

快速傅立叶变换

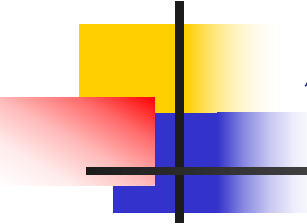


傅立叶变换逆变换

- 目标. 给定n-1次多项式在n个点 $\omega^0, \omega^1, \dots, \omega^{n-1}$ 处的点值 $y_0, \dots, y_{n-1}$, 给出满足条件的唯一的多项式 $a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$.

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \text{L} & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 & \text{L} & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \text{L} & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \text{L} & \omega^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \text{O} & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \text{L} & \omega^{(n-1)(n-1)} \end{bmatrix}^{-1} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

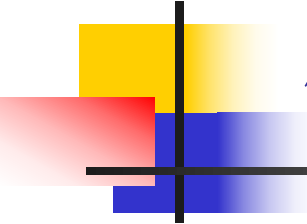
$\uparrow$ Inverse DFT $\uparrow$ Fourier matrix inverse $(F_n)^{-1}$



傅立叶变换逆变换

- 命题：傅立叶变换逆变换矩阵形式如下

$$G_n = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & 1 & \text{L} & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3} & \text{L} & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6} & \text{L} & \omega^{-2(n-1)} \\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9} & \text{L} & \omega^{-3(n-1)} \\ \text{M} & \text{M} & \text{M} & \text{M} & \text{O} & \text{M} \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \omega^{-3(n-1)} & \text{L} & \omega^{-(n-1)(n-1)} \end{bmatrix}$$

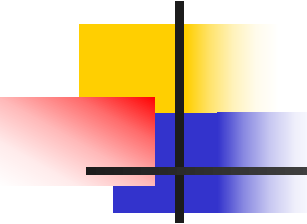


傅立叶变换逆变换

- 发现表达形式与前面的算法计算的对象(DFT)非常类似
- 如何计算傅立叶变换逆变换?

应用同样的算法; 同时注意把
成是 n 次单位根的生成元.

$$\omega^{-1} = e^{-2\pi i / n} \text{ 看}$$

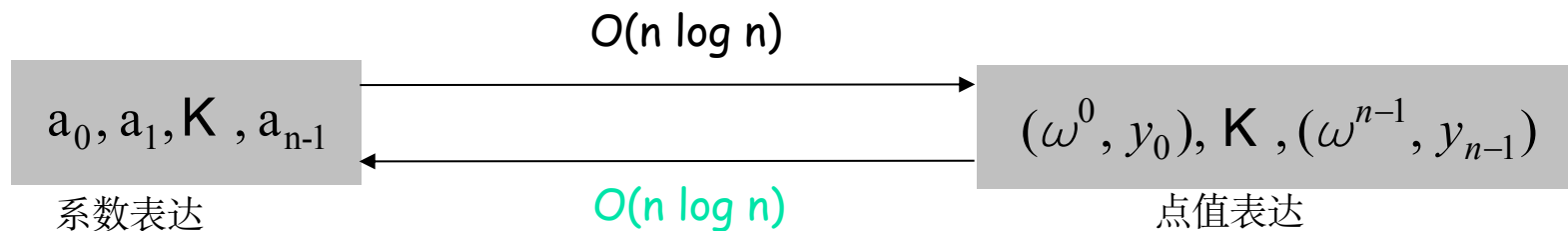


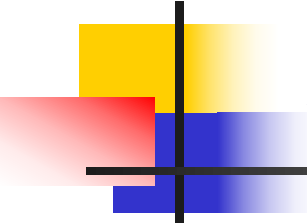
IFFT快速算法

```
ifft(n, a0, a1, ..., an-1) {  
    if (n == 1) return a0  
  
    (e0, e1, ..., en/2-1) ← IFFT(n/2, a0, a2, a4, ..., an-2)  
    (d0, d1, ..., dn/2-1) ← IFFT(n/2, a1, a3, a5, ..., an-1)  
  
    for k = 0 to n/2 - 1 {  
        ωk ← e-2πik/n  
        yk ← (ek + ωk dk) / n  
        yk+n/2 ← (ek - ωk dk) / n  
    }  
  
    return (y0, y1, ..., yn-1)  
}
```

IFFT快速算法

- 定理：IFFT算法能够在 $O(n \log n)$ 步骤内，根据 n 次单位根的值，计算出 $n-1$ 次多项式的系数。（ n 是2的整数次幂）

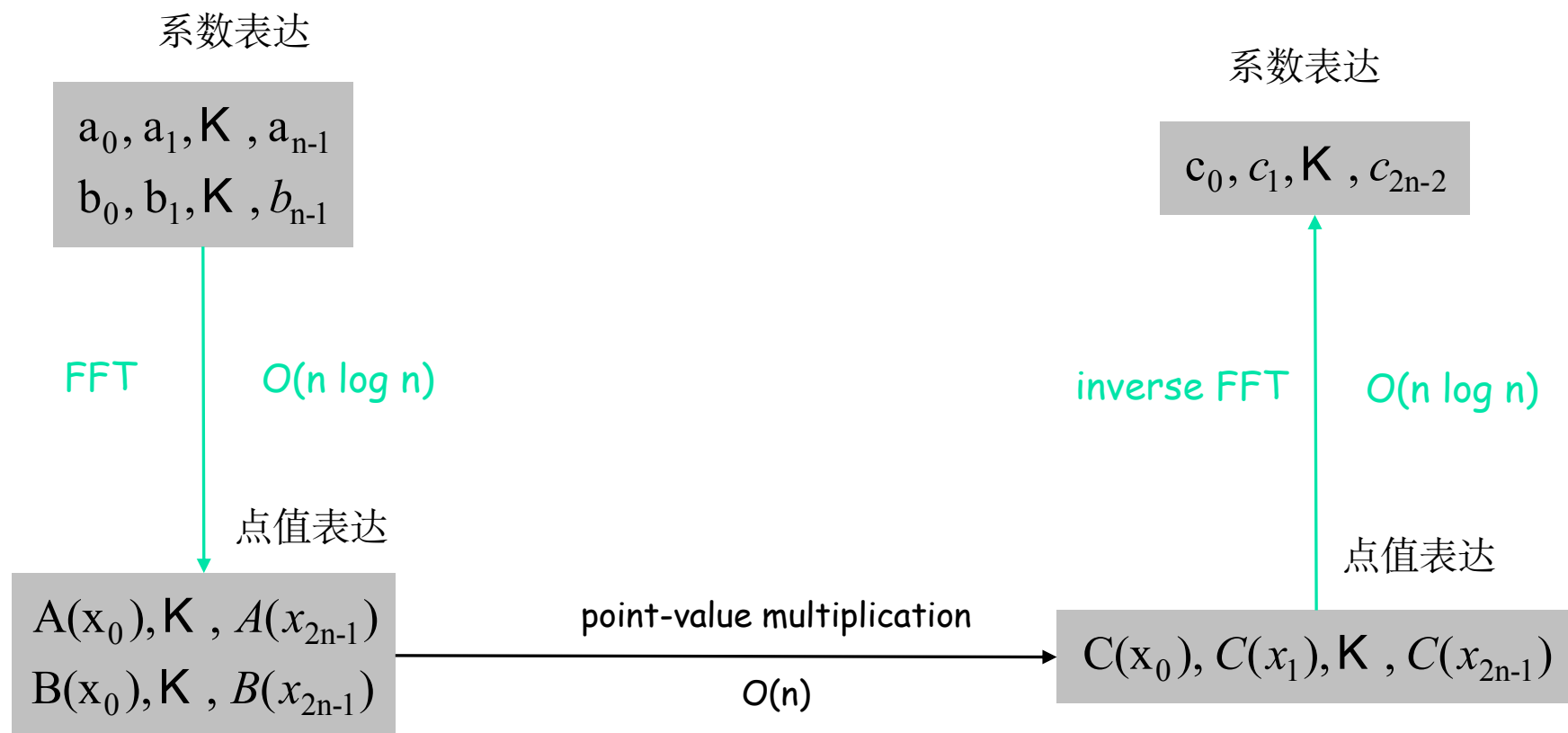




卷积

- 给定向量 $a = (a_0, a_1, \dots, a_{n-1}), b = (b_0, b_1, \dots, b_{n-1})$, 把它们看成多项式 $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$, $B(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$
对于多项式 $C(x) = A(x)B(x)$, 可以知道 $c = (c_0, c_1, \dots, c_{2n-2})$
就是卷积 $a * b$.
- 定理5.15 使用快速傅立叶变换, 可以在 $O(n \log n)$ 时间内计算初始向量 a 和 b 的卷积。

卷积



应用

- 计算两个n位整数的乘法: $b = b_{n-1} \cdots b_1 b_0$ 以及 $a = a_{n-1} \cdots a_1 a_0$

生成两个n次多项式:

注意到 $a = p(10)$, $b = q(10)$.

$$p(x) = \sum_{j=0}^{n-1} a_j x^j$$

$$q(x) = \sum_{j=0}^{n-1} b_j x^j$$

求值 $r(10) = a * b$

- 采用 FFT : $O(n \log n)$.